

# Guida alle prove

Come collaudare le librerie e i banchi di prova pagAmico  
contro il simulatore del pagAmico Dev Kit, dalle IDE

|                   |                                                                                                                                            |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Che cosa si prova | Libreria C#, libreria Kotlin, i due banchi di prova, il collaudo automatico, il proxy di analisi                                           |
| Contro che cosa   | il pagAmico emulato dal Dev Kit di PayPrint, in locale su 127.0.0.1                                                                        |
| Da dove si lancia | Visual Studio per i progetti C#, IntelliJ IDEA per i moduli Kotlin. Tutto quello che serve e' gia' configurato: nessun comando da digitare |
| Quanto dura       | una decina di minuti per il giro completo automatico nei due linguaggi; il resto e' esplorazione a mano                                    |
| Serve la macchina | no. Tutto quello che segue si fa senza il pagAmico in ufficio                                                                              |

## Come leggere questa guida

Il capitolo 2 e' l'indice delle configurazioni pronte: i profili di Visual Studio e le configurazioni di IntelliJ sono **numerati nell'ordine in cui si eseguono**, quindi il menu a tendina dell'IDE e' la sequenza delle prove, dall'alto verso il basso. I capitoli da 3 a 7 sono quella sequenza spiegata, dall'8 al 12 gli strumenti e i casi particolari. Il **13** sta a se': sono le prove che mancano per innestare il pagAmico nel gestionale: quelle a cui PayPrint non ha ancora risposto vanno fatte prima di scrivere il codice che le usa. In appendice gli equivalenti da riga di comando, per quando serve automatizzare.

# 1. Preparazione

Serve una volta sola: due soluzioni da aprire, una per linguaggio.

| IDE           | Che cosa si apre                                                                        | Al primo avvio                                                                                                                                                          |
|---------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Visual Studio | il file<br><code>csharp\PayPrint.PagAmico.sln</code>                                    | ripristina i pacchetti NuGet da solo. Compilare la soluzione una volta ( <b>Compila &gt; Compila soluzione</b> ) per essere sicuri che i sette progetti stiano in piedi |
| IntelliJ IDEA | la <b>cartella</b> <code>kotlin</code> , non un file: la riconosce come progetto Gradle | avvia la sincronizzazione Gradle e scarica le dipendenze. La prima volta ci vuole qualche minuto, le successive sono immediate                                          |

Servono inoltre il **pagAmico Dev Kit** di PayPrint (contiene il simulatore, si installa con `pagAmico-DevKit-Setup-1.0.0.exe` e finisce in `C:\Program Files\pagAmico Dev Kit`), il **.NET 8 SDK** (già presente se Visual Studio 2022 è aggiornato) e un **JDK 17**, che IntelliJ scarica da solo se manca.

## 1.1 I sette progetti C#

| Progetto                                | A che serve                        | Si avvia?            |
|-----------------------------------------|------------------------------------|----------------------|
| <code>PayPrint.PagAmico</code>          | la libreria                        | no, è una libreria   |
| <code>PayPrint.PagAmico.Tests</code>    | 168 test offline, solo 127.0.0.1   | sì, nessun argomento |
| <code>PayPrint.PagAmico.WinForms</code> | il banco di prova                  | sì, nessun argomento |
| <code>PayPrint.PagAmico.LiveTest</code> | collaudo automatico, 64 passi      | sì, 7 profili        |
| <code>PayPrint.PagAmico.Fill</code>     | riempie le giacenze del simulatore | sì, 4 profili        |
| <code>PayPrint.PagAmico.Tap</code>      | proxy che registra il traffico     | sì, 3 profili        |
| <code>PayPrint.PagAmico.Demo</code>     | esempio d'uso minimo               | sì, 2 profili        |

## 1.2 I due moduli Kotlin

| Modulo                        | A che serve                                            | Si avvia?            |
|-------------------------------|--------------------------------------------------------|----------------------|
| <code>pagamico-lib</code>     | la libreria, più i test offline, il collaudo e la demo | sì, 3 configurazioni |
| <code>pagamico-desktop</code> | il banco di prova Compose for Desktop                  | sì, 1 configurazione |

## 2. Le configurazioni già pronte

Nessuna delle prove chiede di scrivere argomenti a mano: sono tutti salvati nel repository, quindi compaiono nell'IDE appena si apre il progetto. I nomi sono numerati nell'ordine in cui si eseguono.

### 2.1 Visual Studio: i profili di avvio

Ogni progetto C# che vuole argomenti ha un file `Properties\launchSettings.json`. Visual Studio li mostra nel **menu a tendina accanto al tasto verde di avvio**: si sceglie il profilo, si preme F5 e gli argomenti sono quelli. Per cambiare progetto: tasto destro sul progetto in Esplora soluzioni, **Imposta come progetto di avvio**.

| Progetto | Profilo                                            | Argomenti                                                    |
|----------|----------------------------------------------------|--------------------------------------------------------------|
| LiveTest | 1 - Collaudo simulatore (completo)                 | 127.0.0.1 9100                                               |
|          | 2 - Collaudo simulatore (solo base)                | 127.0.0.1 9100 base                                          |
|          | 3 - Collaudo simulatore (incassi ed erogazioni)    | 127.0.0.1 9100 collect,erogazione                            |
|          | 4 - Collaudo simulatore (display e stampa)         | 127.0.0.1 9100 display,display2,print,print2                 |
|          | 5 - Collaudo simulatore (sonda DI)                 | 127.0.0.1 9100 di                                            |
|          | 6 - Collaudo macchina reale                        | 192.168.1.231 9100                                           |
|          | 7 - Collaudo macchina reale (solo base)            | 192.168.1.231 9100 base                                      |
| Fill     | 1 - Riempi simulatore (predefinito)                | 127.0.0.1 9100                                               |
|          | 2 - Riempi simulatore e allinea fondo cassa        | 127.0.0.1 9100 --monete 40 --banconote 1200 --aggiorna-fondo |
|          | 3 - Solo monete                                    | 127.0.0.1 9100 --monete 40 --banconote 0                     |
|          | 4 - Solo banconote a 1200                          | 127.0.0.1 9100 --monete 0 --banconote 1200                   |
| Tap      | 1 - Proxy verso il simulatore (9200 -> 9100)       | --listen 9200 --target 127.0.0.1:9100                        |
|          | 2 - Proxy verso il simulatore con dump esadecimale | --listen 9200 --target 127.0.0.1:9100 --hex                  |
|          | 3 - Proxy verso la macchina reale                  | --listen 9200 --target 192.168.1.231:9100                    |
| Demo     | 1 - Demo simulatore                                | 127.0.0.1 9100 1,50                                          |
|          | 2 - Demo macchina reale                            | 192.168.1.231 9100 1,50                                      |

**Tests e WinForms** non hanno profili perché non vogliono argomenti: basta impostarli come progetto di avvio e premere F5.

### 2.2 IntelliJ IDEA: le configurazioni di esecuzione

Sono nella cartella `kotlin\run` e IntelliJ le carica da solo. Compaiono nel **menu a tendina in alto a destra**, accanto al tasto verde di avvio.

| Configurazione               | Che cosa lancia                            |
|------------------------------|--------------------------------------------|
| 1 - Test offline (168)       | i 168 test offline della libreria Kotlin   |
| 2 - Collaudo simulatore      | i 64 passi contro 127.0.0.1:9100           |
| 3 - Banco di prova (Compose) | l'applicazione desktop con tutti i comandi |
| 4 - Collaudo macchina reale  | i 64 passi contro 192.168.1.231:9100       |

Sono quattro, non una per combinazione: le configurazioni di IntelliJ stanno tutte nello stesso menu, quindi ogni voce in piu' e' rumore. Per provare un gruppo solo si modificano gli argomenti della **2**, aggiungendo i gruppi come terzo argomento separati da virgola (`--args="127.0.0.1 9100 collect, erogazione"`); per la demo si cambia la classe di avvio in `-PmainClass=it.payprint.pagamico.demo.DemoKt`. In Visual Studio i profili sono piu' numerosi perche' stanno sotto il rispettivo progetto: non si vedono finche' non si seleziona quello.

#### **Se l'IP della macchina non e' 192.168.1.231**

E' un segnaposto. Il giorno in cui arriva il pagAmico si legge l'indirizzo vero a display, in *Menu Servizi*, e si corregge una volta nei profili: in Visual Studio da **Debug > Proprieta' di debug**, in IntelliJ modificando la configurazione. Da quel momento resta salvato.

### 3. Passo 1: i test offline

Si fanno per primi perché non serve niente acceso: confrontano le stringhe generate dalla libreria con gli esempi **letterali** dei manuali. Durano un paio di secondi. Se qui qualcosa fallisce è inutile passare al simulatore: il difetto è nella composizione dei comandi, non nella comunicazione.

|               | Che cosa fare                                                                                                                                                  | Atteso                                        |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| Visual Studio | tasto destro su <b>PayPrint.PagAmico.Tests</b> ,<br><b>Imposta come progetto di avvio</b> , poi <b>Ctrl+F5</b><br>(senza debug, così la finestra resta aperta) | 168 test superati, 0 falliti                  |
| IntelliJ IDEA | configurazione <b>1 - Test offline (168)</b> , tasto verde                                                                                                     | lo stesso conteggio nella finestra <i>Run</i> |

Vanno rilanciati dopo ogni modifica alla libreria, prima di qualsiasi altra prova.

### 4. Passo 2: accendere il simulatore

Aprire `pagamico_devkit.exe`, andare nella sezione **Simulatore** e premere **Avvia**. Il Dev Kit apre un socket TCP sulla porta **9100** e da quel momento si comporta come un `pagAmico`: accetta i comandi documentati, risponde con gli stessi JSON e aggiorna le giacenze a ogni operazione.

Lasciarlo aperto per tutta la sessione: chiudendolo cadono tutte le connessioni.

#### 4.1 Verifica veloce che sia in ascolto

Dal terminale integrato dell'IDE (in Visual Studio **Visualizza > Terminale**, in IntelliJ la linguetta *Terminal* in basso):

```
netstat -ano | findstr :9100
```

Una riga in stato LISTENING vuol dire che il simulatore è pronto. Nessuna riga: non è partito, ricontrollare il Dev Kit.

#### Se la porta 9100 è occupata

La 9100 è la porta di stampa RAW standard: capita che sia già presa da un driver di stampante. In quel caso si sceglie un'altra porta nel simulatore, e la si corregge nei profili di avvio al posto di 9100. Sulla macchina vera il manuale consiglia comunque di spostarsi, per esempio sulla 43775.

### 5. Passo 3: riempire le giacenze

Il simulatore parte vuoto, e dopo qualche giro di prove si svuota di nuovo. Senza monete l'erogazione del resto smette di funzionare: il collaudo comincia a segnalare `amountUnpaid` valorizzato e `errorList` con la seconda cifra a 9. Non è un difetto della libreria, è la cassa vuota.

#### Prima accendere l'interruttore, o non riempie niente

Dev Kit, sezione *Simulatore*: **Il cliente inserisce l'importo esatto**. Se è spento, il cliente virtuale paga sempre col taglio successivo e la macchina deve erogare il resto. Il ciclo dei tagli si chiude allora a saldo zero: cinque giri accumulano due euro in monete e il sesto, pagato con una banconota da 5, li restituisce tutti come resto. Sessanta giri per non spostare nulla. Con l'interruttore acceso non c'è resto e l'incassato resta tutto dentro.

|               | Che cosa fare                                                                                                                                      |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
|               | Dev Kit, sezione <i>Simulatore</i> : accendere <b>Il cliente inserisce l'importo esatto</b>                                                        |
| Visual Studio | chiudere la connessione del banco ( <b>Disconnetti</b> ): il simulatore serve un client per volta, altrimenti Fill non riesce nemmeno a collegarsi |

| Che cosa fare |                                                                                                                        |
|---------------|------------------------------------------------------------------------------------------------------------------------|
| Visual Studio | progetto di avvio <b>PayPrint.PagAmico.Fill</b> , profilo <b>2 - Riempi simulatore e allinea fondo cassa</b> , Ctrl+F5 |
| IntelliJ IDEA | non serve: il riempimento e' solo lato C#, e riempie la stessa macchina che poi proverà anche il collaudo Kotlin       |

Dura qualche minuto: riempie eseguendo **incassi** ripetuti, alternando i tagli (0,05 / 0,10 / 0,20 / 0,50 / 1,00 / 2,00 per le monete, 5 / 10 / 20 / 50 per le banconote) così la macchina si ritrova spiccioli di ogni valore e riesce a comporre qualsiasi resto. E' quello che porta `errorList` da E0910 a E0110. Il profilo 2 chiude mandando AF, che riallinea il fondo cassa contabile alla giacenza reale: durante le prove le due grandezze si scollano.

#### Mai contro una macchina vera

PagAmicoFill riempie eseguendo incassi reali. Sul simulatore li completa un cliente virtuale; su una macchina vera aprirebbe sessioni di incasso a ripetizione. Per questo non esiste un profilo che punti a un indirizzo diverso da 127.0.0.1, e non va aggiunto.

## 5.1 Perché non si usano le sessioni di ricarica

Sarebbe la strada naturale, ma sul simulatore non funziona: RC, RS, RM e RB aprono regolarmente la sessione, ma non entra nulla, perché sulla macchina vera il contante lo inserisce fisicamente l'operatore e il cliente virtuale del simulatore si attiva solo sugli incassi. Verificato: RS seguito da FR lascia le giacenze identiche.

## 6. Passo 4: il collaudo automatico

E' la prova che conta: 64 passi in sequenza che esercitano **tutti** i comandi implementati, con esito per ciascuno. Va lanciato una volta per linguaggio.

| Che cosa fare |                                                                                                                   |
|---------------|-------------------------------------------------------------------------------------------------------------------|
| Visual Studio | progetto di avvio <b>PayPrint.PagAmico.LiveTest</b> , profilo <b>1 - Collaudo simulatore (completo)</b> , Ctrl+F5 |
| IntelliJ IDEA | configurazione <b>2 - Collaudo simulatore</b> , tasto verde                                                       |

### 6.1 Come si legge l'esito

Ogni passo si annuncia mentre viene eseguito, e alla fine c'e' il riepilogo:

```
--- [B1] ST stato macchina
=> OK monete 41,75 EUR, banconote 1200 EUR, 96 movimenti

=====
OK [B1] ST stato macchina monete 41,75 EUR, banconote 1200 EUR
OK [C3] IN annullo dal client annullato, incassato 0,50, reso 0,50
OK [P2] P2 erogazione taglio scelto erogate 1 banconote da 10 EUR
...

64 passi riusciti, 0 falliti
```

| Esito                        | Significato                                                                                                                                                                                                    |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 64 passi riusciti, 0 falliti | tutto a posto. Il processo esce con codice 0. Sono i passi dei 13 gruppi di default: nel sorgente le chiamate sono 49, ma tre gruppi girano in ciclo. Con il profilo della sonda DI e i riavvii si arriva a 71 |
| qualche FAIL                 | la riga porta il tipo di eccezione e il messaggio. Prima di sospettare la libreria, guardare il capitolo 10: quasi sempre e' la cassa vuota o il simulatore rimasto occupato                                   |
| CONNESSIONE FALLITA          | il simulatore non e' partito, oppure e' su un'altra porta                                                                                                                                                      |

Il collaudo scrive anche un file di log completo, e ne stampa il percorso come prima riga: Log della sessione: ...\\collaudo-2026-09-03.log.

### 6.2 Provare un gruppo solo

Il giro completo dura qualche minuto. Quando si sta lavorando su un'area sola conviene un gruppo alla volta: in Visual Studio sono i profili dal 2 al 5, in IntelliJ si modificano gli argomenti della configurazione 2. Sotto, che cosa copre ciascun gruppo.

| Gruppo     | Copre                                              |
|------------|----------------------------------------------------|
| base       | ST e decodifica delle giacenze, CL, LO             |
| cash       | SM, SB, EM, EB, AF, BT                             |
| collect    | IN con parziali, annullo dal client, CM, PA        |
| erogazione | P2, PM, M2, MF, AZ, I2, IM                         |
| ricariche  | RM, R3, RB, R2, RS, VC, VS, ciascuna chiusa con FR |
| display    | DT/DS, DM/DC, QR/QA, TS/ID/CO                      |
| display2   | DG, DI con tastiera numerica                       |
| print      | PTSTAT e uno scontrino completo da PTSTST a PTSTEN |

| Gruppo    | Copre                                                                          |
|-----------|--------------------------------------------------------------------------------|
| print2    | PTITON/OF, PTDBON/OF, PTUN, PTFB, PTCP, PTPRWR, PTPRRE, PTBCBC, PTPRDT, PTSTAN |
| system    | PR, PL, PO, RC/FR, e un comando inesistente per verificare CMD ERROR           |
| pos2      | PLT con ristampa, PS, PP                                                       |
| immagini  | SI, SR, SF, e il confronto fra i due incapsulamenti binari                     |
| movimenti | MV, MI                                                                         |
| di        | sonda le quattro varianti plausibili del comando DI                            |
| riavvii   | PZ e RI. Esclusi di default: riavviano POS e macchina                          |

## 7. Passo 5: le prove a mano dai banchi

I due banchi di prova sono gemelli: stesse nove schede, stessi comandi, stesso pannello del traffico. Servono per provare un singolo comando, riprodurre un caso segnalato, o vedere coi propri occhi che cosa risponde la macchina.

| Che cosa fare |                                                                                                                                           |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Visual Studio | tasto destro su <b>PayPrint.PagAmico.WinForms</b> , <b>Imposta come progetto di avvio</b> , poi F5. Non ha profili: non vuole argomenti   |
| IntelliJ IDEA | configurazione <b>3 - Banco di prova (Compose)</b> , tasto verde. La prima volta Gradle compila anche il modulo desktop, poi e' immediato |

### 7.1 Collegarsi

Scheda **Connessione**, pulsante **Simulatore locale**: imposta 127.0.0.1 e 9100 e connette in un colpo solo. Il pallino in alto diventa verde. Per la macchina vera si scrivono IP e porta a mano e si preme **Connetti**.

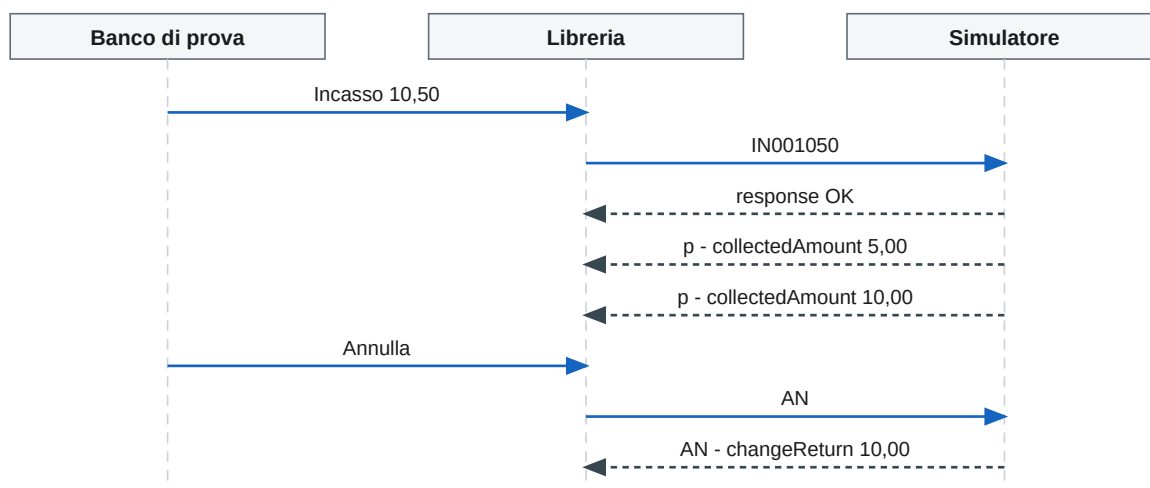
Sempre in questa scheda ci sono le due opzioni di protocollo: il **terminatore comandi** (vuoto = nessuno, verificato solo sul simulatore. PayPrint chiede CR o CR+LF, nella casella \r o \r\n: sulla macchina va messo, sul simulatore e' ancora da provare) e la **password** richiesta dai comandi che muovono denaro (PA, P2, PM, M2, MF, BT, AF, AZ).

### 7.2 Le nove schede

| Scheda      | Che cosa ci si fa                                                                                                                                              |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Connessione | collegamento, opzioni di protocollo, percorso del file di log                                                                                                  |
| Incasso     | IN, I2, PO, IM, con annullo e commit                                                                                                                           |
| Contanti    | giacenze, travasi, azzeramenti, soglie                                                                                                                         |
| Ricariche   | le sessioni di carico e scarico, ciascuna da chiudere con FR                                                                                                   |
| POS         | pagamenti con carta, ristampe, stato del terminale                                                                                                             |
| Display     | messaggi, QR, liste, finestra di input, immagini                                                                                                               |
| Stampa      | lo scontrino completo e ogni singolo comando PT, diviso in stile, font e layout, contenuto, servizio                                                           |
| Sistema     | stato, reset, riavvii, movimenti                                                                                                                               |
| Console     | invio libero di una stringa qualsiasi, con cronologia sulle frecce su/giu'. E' la scheda che serve per provare un comando non ancora previsto dall'interfaccia |

### 7.3 Uno scenario completo da provare

Il piu' istruttivo e' un incasso interrotto a meta': mostra i parziali, l'annullo e il resto, cioe' i tre punti dove un'integrazione fatta male si rompe.



Un incasso annullato: la macchina restituisce quanto aveva già preso.

| Pas so | Che cosa fare                                                       | Che cosa deve succedere                                                                                                                                                                                                                                                            |
|--------|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1      | scheda <b>Incasso</b> , importo 10,50, premere <b>Avvia incasso</b> | nel traffico compare IN001050 e subito dopo response OK                                                                                                                                                                                                                            |
| 2      | attendere qualche secondo                                           | arrivano messaggi p con collectedAmount che cresce: e' il cliente virtuale che infila denaro                                                                                                                                                                                       |
| 3      | premere <b>Annulla</b> prima che arrivi a 10,50                     | parte AN, la macchina rende quanto raccolto e chiude con response AN e l'importo reso in changeReturn. Se invece arriva response IN l'annullo e' stato <b>tardivo</b> : il cliente virtuale aveva già completato, e va alzato <i>Ritardo fra un pezzo e l'altro</i> nel simulatore |
| 4      | scheda <b>Contanti</b> , premere <b>Stato</b>                       | le giacenze sono tornate come prima: quello che era entrato e' uscito                                                                                                                                                                                                              |

#### Il passo che conta davvero

Il terzo. Se l'annullo non parte, la macchina resta occupata e ogni comando successivo risponde E100 con errorType 99. E' esattamente il difetto che il collaudo ha fatto emergere sulla libreria Kotlin, dove la cancellazione veniva scambiata per un timeout e AN non veniva mai trasmesso.

## 8. Guardare i log

Tre livelli, attivabili indipendentemente. Il primo dice *che cosa* e' passato sul filo, il secondo *perche'* e' andata come e' andata, il terzo li conserva.

| Livello     | Dove si accende                                     | Che cosa registra                                                                                                                                                                    |
|-------------|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Traffico    | sempre attivo, pannello in basso nei banchi         | le stringhe inviate e i messaggi ricevuti, cosi' come sono                                                                                                                           |
| Diagnostica | casella <b>diagnostica libreria</b>                 | connessione, pause imposte fra un invio e l'altro, byte letti dal socket, separazione dei messaggi, resto rimasto nel buffer, attese soddisfatte o scadute, messaggi non sollecitati |
| File        | casella <b>registra su file</b> , attiva di default | tutto quanto sopra, un file al giorno, in append, apribile mentre viene scritto                                                                                                      |

### 8.1 Dove finiscono i file

```
%LOCALAPPDATA%\PayPrint.PagAmico\logs
```

Un file per programma e per giorno, cosi' i due banchi possono girare insieme senza mescolare i tracciati: winforms-2026-09-03.log, compose-..., collaudo-..., demo-..., tap-.... Nella scheda **Connessione** c'e' il percorso del file corrente, e il pulsante **Apri cartella log** la apre in Esplora risorse.

### 8.2 Farsi rileggere una sessione

Nella cartella strumenti c'e' `analizza_log.py`: legge i file della giornata e dice se la sessione e' andata bene o dove si e' rotta. Cerca i comandi rimasti senza risposta (esclusi quelli che per protocollo non rispondono e quelli che rispondono solo su azione dell'utente), gli invii troppo ravvicinati, gli errori con la spiegazione, lo stato delle scorte, le attese scadute, e chiude con l'elenco dei comandi che in quella sessione non sono mai stati provati.

```
python strumenti/analizza_log.py i log di oggi
python strumenti/analizza_log.py --data 2026-09-03 di un giorno preciso
```

### 8.3 Che cosa si vede con la diagnostica accesa

```
.. connesso a 127.0.0.1:9100 (pausa minima fra invii 80 ms, terminatore nessuno)
.. attesa di 80 ms prima dell'invio: il pagAmico ignora i comandi troppo ravvicinati
.. in attesa dell'esito di 'ST' (timeout 15s)
.. letti 1076 byte dal socket
.. messaggio separato: JSON, 1076 caratteri
.. attesa di 'ST' soddisfatta da: {"response":"ST", ...
```

Le righe che contano sono le ultime due **quando non compaiono**. Se la traccia si ferma su *in attesa* e poi dice *attesa scaduta* senza aver letto un solo byte, il comando non e' mai arrivato alla macchina: il problema non e' nell'interpretazione della risposta ma nell'invio. Le prime due righe sono quelle della libreria di oggi, senza terminatore e con 80 ms di pausa: PayPrint dice che con CR o CR+LF la pausa non serve, e finche' non e' provato resta come rete di sicurezza.

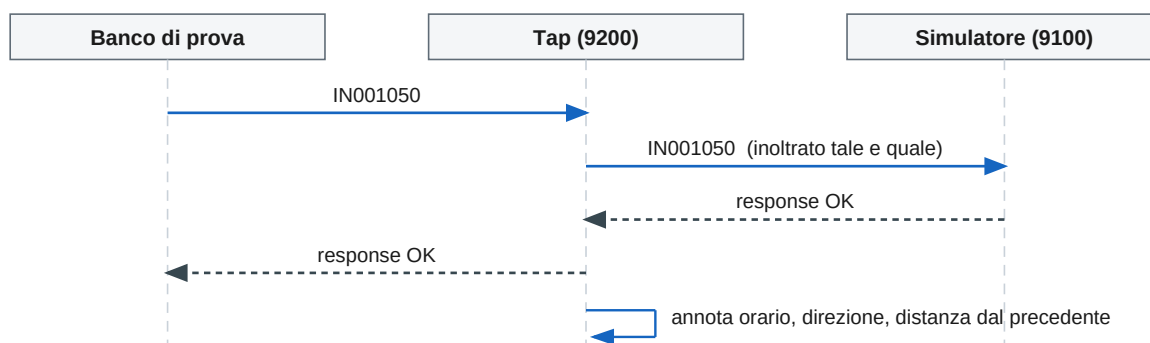
Nei programmi da riga di comando (collaudo, demo) la diagnostica e' sempre accesa e compare nella finestra di output dell'IDE, con il prefisso . . .

### 8.4 La finestra separata

Il pulsante **Finestra log** apre il traffico in una finestra a se', con filtri per direzione, ricerca testuale, scorrimento automatico ed esportazione. Serve quando si vuole tenere d'occhio il traffico su un monitor e usare il banco di prova sull'altro.

## 9. Il proxy di analisi

Il pannello del traffico mostra i messaggi *logici*, già ricomposti dalla libreria. Il proxy invece registra **ogni segmento TCP così com'è arrivato**: è l'unico modo per vedere due comandi partiti troppo vicini, o una risposta spezzata in più pezzi.



Il proxy si inserisce fra client e macchina senza modificare un byte.

### 9.1 Avviarlo insieme al banco di prova

Il proxy e il banco devono girare contemporaneamente, quindi servono due processi. In Visual Studio si fa dalle proprietà della soluzione, senza aprire una seconda istanza:

| # | Che cosa fare                                                                                                                   |
|---|---------------------------------------------------------------------------------------------------------------------------------|
| 1 | tasto destro sulla <b>soluzione</b> in Esplora soluzioni, <b>Proprietà'</b>                                                     |
| 2 | <b>Progetto di avvio</b> , opzione <b>Piu' progetti di avvio</b>                                                                |
| 3 | mettere <b>Avvia</b> su <b>PayPrint.PagAmico.Tap</b> e su <b>PayPrint.PagAmico.WinForms</b> , <b>Nessuno</b> su tutti gli altri |
| 4 | F5: partono entrambi. Nel banco, scheda Connessione, scrivere porta <b>9200</b> invece di 9100 e premere <b>Connetti</b>        |

Per il banco Kotlin la cosa è anche più semplice: si avvia il proxy da Visual Studio (profilo **1 - Proxy verso il simulatore**) e il banco da IntelliJ, sono due IDE distinte. Poi porta 9200 nel banco.

### 9.2 Che cosa mostra

```
15:57:41.892 C->M [#1]          2 byte ST
              [#1] risposta a 'ST' dopo 34 ms
15:57:42.447 C->M [#2]          2 byte DS
15:57:42.447 C->M [#2] + 0,001s    6 byte PTSTAT
15:57:42.448 !! [#2] ATTENZIONE: comando inviato solo 1 ms dopo il precedente.
              Il pagAmico legge il buffer del socket in un colpo solo: sotto i
              30 ms il secondo comando può essere ignorato senza risposta.
15:57:44.472 -- [#2] sessione chiusa. client 2 segmenti / 8 byte,
              macchina 0 segmenti / 0 byte.
```

L'ultima riga è la prova del difetto: la macchina, qui il simulatore, non ha risposto affatto a comandi mandati senza terminatore. PayPrint attribuisce il fenomeno al simulatore e dice che con CR o CR+LF la pausa non serve: è ancora da provare. A fine sessione il proxy calcola minimo, media e massimo sia delle pause del client sia dei tempi di risposta della macchina.

### 9.3 L'uso più utile: osservare il Dev Kit

Il Dev Kit di PayPrint non è solo il simulatore: è anche un client. Impostando **9200** come porta dentro il Dev Kit, si vede esattamente che cosa manda l'applicazione del costruttore, comprese le pause che tiene fra un comando e l'altro. È la risposta autorevole ai punti su cui i manuali tacciono o si contraddicono: il formato del comando DI, quale dei due incapsulamenti delle immagini sia quello giusto, se dopo i pacchetti

immagine serva il terminatore. Per i comandi il terminatore l'ha chiarito PayPrint: CR o CR+LF. Il profilo **2** aggiunge il dump esadecimale, indispensabile per l'invio delle immagini.

## 10. Quando qualcosa non va

Quasi tutti i fallimenti hanno una di queste sei cause, e nessuna e' un difetto della libreria.

| Sintomo                                                                                | Causa probabile                                                                                 | Come si verifica e si risolve                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Connessione rifiutata                                                                  | simulatore non avviato, o porta diversa                                                         | <code>netstat -ano   findstr :9100</code> dal terminale dell'IDE. Riavviare il Dev Kit                                                                                                                                                                           |
| Ogni comando risponde E100 con <code>errorType 99</code>                               | la macchina e' rimasta occupata da un'operazione precedente non chiusa                          | mandare AN dalla scheda Console del banco, o FR se era aperta una sessione di ricarica. Se non basta, riavviare il simulatore                                                                                                                                    |
| <code>amountUnpaid</code> valorizzato, <code>errorList</code> con la seconda cifra a 9 | monete finite: il resto non e' componibile                                                      | rilanciare il profilo <b>2 di Fill</b> (capitolo 5)                                                                                                                                                                                                              |
| Un passo del collaudo fallisce con <code>QTABANCONOTE</code>                           | il taglio richiesto non e' piu' in macchina, di solito perche' un passo precedente l'ha erogato | e' il motivo per cui i passi sulle banconote leggono ST prima di scegliere il taglio. Se ricapita, ricaricare e rilanciare                                                                                                                                       |
| Un comando non riceve risposta, l'attesa scade                                         | comando senza terminatore, inviato troppo presto dopo il precedente                             | accendere la <b>diagnostica libreria</b> : se dice <i>attesa scaduta</i> senza aver letto byte, e' questo. Oppure mettere il proxy in mezzo e cercare la riga ATTENZIONE. Il rimedio indicato da PayPrint e' il terminatore CR (capitolo 7.1), ancora da provare |
| Nessuna risposta, ma il comando e' corretto                                            | e' uno dei comandi che per protocollo non rispondono                                            | CL, DS, DC, QA, CO, DT, DG, TS non prevedono risposta. Il promemoria e' anche nella scheda Connessione                                                                                                                                                           |

### 10.1 Problemi dell'IDE, non del protocollo

| Sintomo                                                         | Rimedio                                                                                                                                                                          |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| La finestra del programma si chiude subito e non si legge nulla | in Visual Studio usare <b>Ctrl+F5</b> (avvia senza debug): tiene la finestra aperta fino a un tasto. Con F5 si chiude alla fine del processo                                     |
| I profili non compaiono nel menu di avvio                       | il progetto selezionato non e' quello di avvio, oppure la soluzione non e' stata ricaricata. I profili stanno in <code>Properties\launchSettings.json</code> di ciascun progetto |
| Le configurazioni non compaiono in IntelliJ                     | e' stata aperta la cartella sbagliata: va aperta <code>kotlin</code> , quella che contiene <code>settings.gradle.kts</code> e la cartella <code>.run</code>                      |
| IntelliJ segnala errori di compilazione appena aperto           | la sincronizzazione Gradle non e' finita. Attendere, o forzarla dall'icona di ricarica nel pannello Gradle                                                                       |

#### Prima di segnalare un difetto

Allegare il file di log della sessione (`%LOCALAPPDATA%\PayPrint.PagAmico\logs`) con la **diagnostica accesa**. Senza quello, dal solo traffico non si distingue un comando mai partito da una risposta mai arrivata.

## 11. Riepilogo: la sequenza completa

Da eseguire nell'ordine. La colonna dell'IDE dice esattamente che cosa selezionare.

| # | Passo                   | Visual Studio                                                              | IntelliJ IDEA                |
|---|-------------------------|----------------------------------------------------------------------------|------------------------------|
| 1 | test offline            | progetto <b>Tests</b> , Ctrl+F5                                            | config. <b>1</b>             |
| 2 | accendere il simulatore | Dev Kit, sezione Simulatore, <b>Avvia</b>                                  | -                            |
| 3 | riempire le giacenze    | progetto <b>Fill</b> , profilo <b>2</b>                                    | -                            |
| 4 | collaudo completo       | progetto <b>LiveTest</b> , profilo <b>1</b>                                | config. <b>2</b>             |
| 5 | prove a mano            | progetto <b>WinForms</b> , F5                                              | config. <b>3</b>             |
| 6 | se qualcosa non torna   | diagnostica accesa nel banco; poi <b>Tap</b> profilo <b>1</b> e porta 9200 | diagnostica accesa nel banco |

Atteso al termine: **168 su 168** offline e **64 su 64** di collaudo, in entrambi i linguaggi.

## 12. Il giorno in cui arriva la macchina

Cambia pochissimo, ed e' voluto: il simulatore e la macchina parlano lo stesso protocollo sulla stessa porta. I profili per la macchina reale ci sono gia'.

| Cosa              | Con il simulatore                | Con la macchina                                                                                                            |
|-------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| profilo da usare  | profili 1-5 (LiveTest)           | profilo <b>7 - Collaudo macchina reale (solo base)</b> per primo, poi il <b>6</b> . In IntelliJ la configurazione <b>4</b> |
| indirizzo         | 127.0.0.1                        | l'IP del pagAmico, leggibile a display in <i>Menu Servizi</i> . Da correggere una volta nei profili                        |
| porta             | 9100                             | 9100 di fabbrica, da spostare (il manuale suggerisce 43775)                                                                |
| riempimento       | profilo 2 di <b>Fill</b>         | <b>mai</b> . Il contante lo carica l'operatore, con una sessione di ricarica                                               |
| gruppo riavvii    | escluso di default               | escluso, salvo prova voluta: riavvia POS e macchina                                                                        |
| terminatore       | nessuno; CR e' ancora da provare | <b>CR</b> (o CR+LF), come chiede PayPrint                                                                                  |
| pausa fra comandi | 80 ms, misura empirica           | PayPrint la dice non necessaria con il terminatore CR o CR+LF. Resta a 80 ms come rete di sicurezza finche' non e' provato |

### Le prime tre cose da fare sulla macchina vera

1. Avviare il **Tap** col profilo **3 - Proxy verso la macchina reale** fin dalla prima connessione: il tracciato della prima sessione vale piu' di qualsiasi prova successiva. 2. Cominciare dal profilo **7**, solo il gruppo base, non dal collaudo completo. 3. Mettere il terminatore **CR** che chiede PayPrint (nei banchi \r nella casella del capitolo 7.1; il collaudo oggi non ha l'opzione). Gli 80 ms di pausa restano come rete di sicurezza finche' la prova *raffica* di docs/esito-risposta-payprint.md non mostra che con CR i comandi passano anche senza; il valore sta in una sola proprieta' (MinimumCommandInterval in C#, minimumCommandIntervalMs in Kotlin).

## 13. Le prove che mancano per l'innesto in Giano

I 64 passi del collaudo dimostrano che le librerie parlano il protocollo. Non dimostrano che il **flusso di pagamento** del gestionale regga, perche' quello dipende da come si comporta la macchina in situazioni che il simulatore non sa riprodurre. L'analisi dell'innesto in Giano (documento *Integrazione pagAmico*, capitolo 9; l'elenco dei punti aperti e' nella sua sezione 8.2) ha isolato **cinque punti che nessuno ha ancora osservato**.

Per tre (la 1, la 4 e, in parte, la 5) c'e' ora la risposta di PayPrint dell'11 settembre: da osservare diventano da confermare. La 4 era il vincolo che bloccava il disegno del flusso, e la risposta lo scioglie: CM si manda durante l'incasso. Il lavoro che resta e' nelle librerie (docs/esito-risposta-payprint.md, sezione 3). La 2 e la 3 restano da osservare, prima di scrivere il codice che le usa.

### 13.1 Le cinque prove

| Che cosa si vuole vedere                                   | Come si provoca                                                                                                                                                                                                                                                                                                                  | Perche' conta                                                                                                                                                                                                                                                                       |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. che campi porta davvero un <b>parziale</b>              | un incasso qualsiasi, guardando il traffico grezzo col <b>Tap</b> invece dell'esito finale                                                                                                                                                                                                                                       | l'interfaccia mostra al cliente quanto ha inserito, e durante un incasso non si puo' interrogare nulla. Per PayPrint i parziali arrivano a ogni aggiunta di contante e sono <b>cumulativi</b> : un parziale perso non costa nulla, la prova lo conferma                             |
| 2. se sulla risposta di <b>annullo</b> arriva l'incassato  | avviare un incasso, inserire meno del dovuto, annullare                                                                                                                                                                                                                                                                          | senza l'incassato non si sa se la restituzione e' stata <b>incompleta</b> : e' denaro del cliente che resta in macchina senza che il gestionale lo sappia                                                                                                                           |
| 3. una <b>restituzione incompleta</b> vera                 | svuotare le monete dal banco WinForms, scheda <b>Manutenzione</b> (MF, AZ, o soglie a zero con SM), poi chiedere un incasso che richieda resto. <b>Fill non serve</b> : riempie soltanto, si ferma appena la giacenza raggiunge il bersaglio                                                                                     | e' il caso peggiore del flusso attuale, e l'unico per cui il pagAmico offre un campo dedicato invece di una sottrazione                                                                                                                                                             |
| 4. che il commit <b>chiuda</b> davvero un incasso in corso | avviare un incasso e mandare il commit mentre e' aperto                                                                                                                                                                                                                                                                          | la via c'e': per PayPrint durante IN la macchina accetta CM. Dall'11 settembre la libreria manda il commit a incasso aperto e chiude sull'esito invece che sull'accettazione (difetto D1, corretto sul simulatore): la prova verifica che sulla macchina valga lo stesso            |
| 5. che cosa fa la macchina se il <b>client si arrende</b>  | avviare un incasso, staccare la rete a meta' e ricollegarsi <b>dallo stesso IP</b> ; poi chiuderlo con la chiusura forzata dal pannello (gesto e password da chiarire), che torna come AN. Non lasciarlo scadere: PayPrint dice di non mettere timeout, e allo scadere dei 5 minuti la libreria lo abbandona aperto (difetto D2) | per PayPrint dopo IN non c'e' timeout: l'incasso resta aperto finche' non viene chiuso. Da vedere se dopo la riconnessione parziali ed esito arrivano sul nuovo socket, e quanto e' entrato nel frattempo. E' la rete di sicurezza per il PC che si riavvia in mezzo a un pagamento |

#### Perche' il simulatore non basta

Il simulatore non permette di forzare un resto insufficiente, non mostra i parziali come li manda la macchina, e non ha uno stato da cui ripartire dopo una caduta. Le prove 2, 3 e 5 **richiedono una macchina vera**. La 1 e la 4 si possono tentare sul simulatore col Tap acceso, ma il risultato va confermato comunque.

### 13.2 Come registrarle

Tutte e cinque vanno fatte **col Tap acceso** (capitolo 9), non dal solo banco di prova: quello che serve non e' l'esito che la libreria restituisce, ma i byte che la macchina manda davvero, nell'ordine in cui li manda. Il tracciato del proxy si rilegge dal file `tap-AAAA-MM-GG.log`, che porta il riepilogo gia' in fondo; `strumenti/analizza_log.py` serve per il log *di libreria* della stessa sessione e i tracciati del proxy li scarta di proposito.

Le stesse domande, riscritte per PayPrint con il contesto che serve a loro, stanno in `docs/mail-payprint-domande-protocollo.md`: la prova 1 e' il punto 2.7, la 2 il punto 2.2, la 4 il punto 2.1, la 5 il punto 2.4; la prova 3 non ha un punto suo, perche' e' la verifica sul campo di quello che il punto 2.2 chiede a parole. PayPrint ha risposto l'11 settembre (`docs/risposta-payprint-2026-09-11.md`, analisi in `docs/esito-risposta-payprint.md`): la 2.1 e la 2.7 sono chiuse, la 2.4 in parte, quindi le prove 1, 4 e 5 diventano una conferma invece che una scoperta. La 2.2 resta aperta, e con lei la 2 e la 3.

### Una prova da non fare

Nessuna delle cinque va lanciata contro una macchina di terzi, compresa quella che PayPrint offre di collegare da remoto, senza via libera esplicito: la 3 richiede di svuotare le giacenze e la 5 lascia deliberatamente un incasso aperto, che sulla macchina non scade da solo e va chiuso esplicitamente (AN, CM o chiusura forzata dal pannello). Sul **simulatore**, invece, si fa qualsiasi cosa.

## Appendice. Gli stessi passi da riga di comando

Servono solo per automatizzare (una catena di build, una prova rapida senza aprire l'IDE). Vanno dati dalla cartella pagamico-sdk.

```
dotnet run --project csharp/PayPrint.PagAmico.Tests
dotnet run --project csharp/PayPrint.PagAmico.Fill -- 127.0.0.1 9100 `
    --monete 40 --banconote 1200 --aggiorna-fondo
dotnet run --project csharp/PayPrint.PagAmico.LiveTest -- 127.0.0.1 9100
dotnet run --project csharp/PayPrint.PagAmico.WinForms
dotnet run --project csharp/PayPrint.PagAmico.Tap -- --listen 9200 --target 127.0.0.1:9100

cd kotlin
gradle :pagamico-lib:run
gradle :pagamico-lib:run "-PmainClass=it.payprint.pagamico.LiveTestKt" "--args=127.0.0.1 9100"
gradle :pagamico-desktop:run
```

Le virgolette attorno agli interi argomenti servono: senza, PowerShell spezza -PmainClass=... e Gradle riceve un parametro monco. L'apice inverso e' la continuazione di riga di PowerShell.

Prime Software S.r.l. - documento tecnico interno